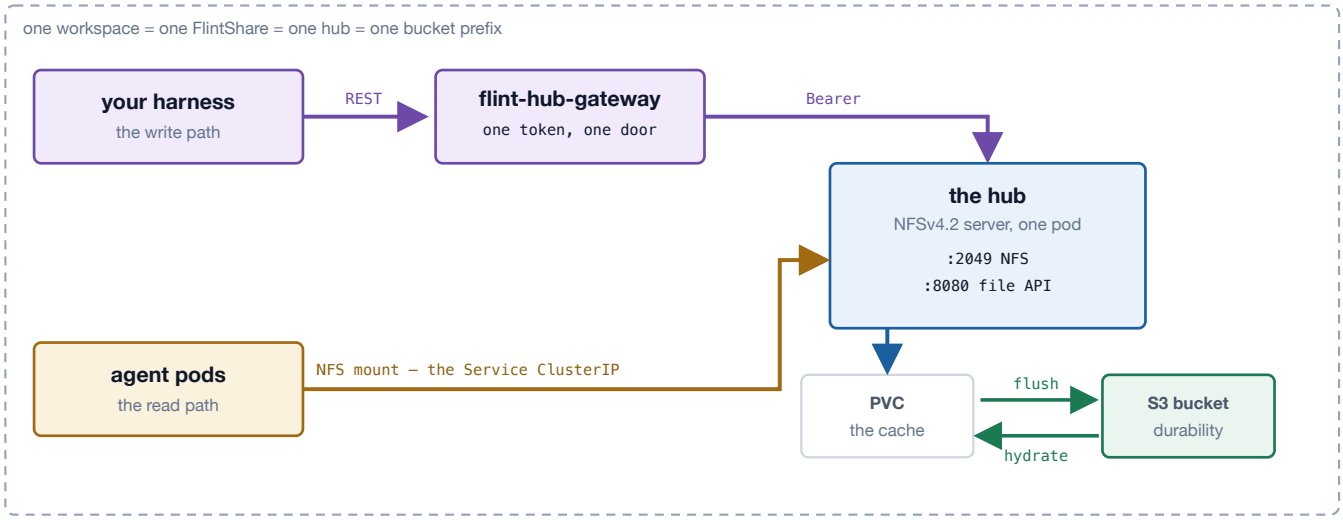
SETUP GUIDE

Flint-lite for agent fleets

A shared POSIX filesystem for agent workspaces, durable in **your own S3 bucket**, that scales to zero when nobody is using it. Your harness writes over REST; your agents read through a mounted PVC that pulls from S3 on first touch.

IMAGES `1.60.0` **CHART** `flint-lite-operator 0.6.2` **BUCKET** `must exist, versioning on` **ON AGENT CLUSTERS** `nothing`



Two doors, one filesystem. The REST API is dispatched in-process through the hub's own NFS compound dispatcher — not a second reader of the export directory — so both doors inherit the same locking, eviction handling and export confinement. **The PVC is a cache and the bucket is the durability story**: a file that is not local yet is fetched from S3 on first touch, and a lost PVC is a rebuild rather than a loss.

BEFORE YOU START

An S3 bucket	Must already exist, **versioning on**. Nothing here creates or deletes buckets.
Kubernetes	1.25+, with `kubectl` and `helm`.
On mounting nodes	An NFS client (`mount.nfs4`). Ships on Amazon Linux 2023, Ubuntu, GKE COS, AKS. Check Bottlerocket and Talos.

1 Install the operator and the gateway

Both come from one chart and one image — the gateway is the same binary with a different command, so enabling it pulls nothing extra.

```
kubectl create namespace flint-system
kubectl -n flint-system create secret generic flint-gateway-token \
  --from-literal=token="$(openssl rand -base64 32)"
kubectl -n flint-system create secret generic flint-gateway-root \
  --from-literal=key="$(openssl rand -base64 48)"

helm install flint-lite-operator \
  oci://registry-1.docker.io/dilipdalton/flint-lite-operator --version 0.6.2 \
  -n flint-system \
  --set gateway.enabled=true \
  --set gateway.tokenSecretRef=flint-gateway-token \
  --set gateway.rootKeySecretRef=flint-gateway-root
```

The chart **refuses to render** without an inbound token, without a hub credential, or with both hub credentials set — a misconfiguration is a `helm` error, not a running open proxy. Put an Ingress with TLS in front of the gateway; `gateway.service.type` is `ClusterIP` by default and should stay that way.

> **THE GATEWAY EXISTS ONLY IN CHART 0.2.6 AND LATER**
>
> Older charts accept `--set gateway.enabled=true` and render nothing, with no error. If the gateway Deployment never appears, check `helm show chart` first.

Credentials, end to end

Four credentials exist and they are not interchangeable. This is the whole set — nothing else needs one.

CREDENTIAL	WHO HOLDS IT	CREATED BY
Bucket keys	the hub	you — MinIO or a real bucket, below
Gateway token	your harness	`openssl rand`, step 1
Gateway root key	the gateway only	`openssl rand`, step 1
Per-share API token	the hub + the gateway	derived, step 3

Dev: MinIO in-cluster

The quickest working setup, and the one every drill in this repo runs against. No cloud account, no IAM.

```
kubectl create ns minio
kubectl -n minio apply -f - <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata: { name: minio }
spec:
  replicas: 1
  selector: { matchLabels: { app: minio } }
  template:
    metadata: { labels: { app: minio } }
    spec:
      containers:
        - name: minio
          image: cgr.dev/chainguard/minio:latest
          args: ["server", "/data"]
          env:
            - { name: MINIO_ROOT_USER,     value: "flintdev" }
            - { name: MINIO_ROOT_PASSWORD, value: "flintdev123" }
          ports: [{ containerPort: 9000 }]
          volumeMounts: [{ name: data, mountPath: /data }]
      volumes: [{ name: data, emptyDir: {} }]
---
apiVersion: v1
kind: Service
metadata: { name: minio }
spec:
  selector: { app: minio }
  ports: [{ port: 9000, targetPort: 9000 }]
EOF

# bucket + versioning, through a port-forward
kubectl -n minio port-forward svc/minio 9000:9000 &
export AWS_ACCESS_KEY_ID=flintdev AWS_SECRET_ACCESS_KEY=flintdev123 AWS_DEFAULT_REGION=us-east-1
aws --endpoint-url http://127.0.0.1:9000 s3 mb s3://flint-dev
aws --endpoint-url http://127.0.0.1:9000 s3api put-bucket-versioning \
  --bucket flint-dev --versioning-configuration Status=Enabled

# the Secret is the same shape everywhere — AWS_* names, verbatim
kubectl -n workspaces create secret generic flint-s3 \
  --from-literal=AWS_ACCESS_KEY_ID=flintdev \
  --from-literal=AWS_SECRET_ACCESS_KEY=flintdev123
```

Then `spec.endpoint` is the only line that differs from a real-S3 share:

```
spec:
  bucket: flint-dev
  endpoint: http://minio.minio.svc:9000   # forces path-style addressing
  region: us-east-1                       # any value MinIO accepts
  credentialsSecretRef: flint-s3
```

A DEV SETUP HIDES THE PERMISSION PROBLEM

MinIO's root user can do everything, so the grants in step 2 are never exercised until you move to a real bucket. That is exactly how the missing `s3:ListBucketMultipartUploads` went unnoticed — every local drill passed. Budget a first-boot failure on your first real bucket, and read the hub's log: it names the action.

Production: a bucket-scoped identity

Create the bucket with versioning on, then an identity scoped to that bucket only carrying the grants listed in step 2. On AWS: `s3api create-bucket` + `put-bucket-versioning`, then `iam create-user` / `put-user-policy` / `create-access-key`; the key pair goes into `flint-s3` under the `AWS_*` names. With **IRSA**, omit `credentialsSecretRef` and annotate the hub's ServiceAccount with a role ARN carrying the same policy. Any S3-compatible store — Ceph RGW, MinIO in production — works identically through `spec.endpoint`.

How your harness gets its token

```
kubectl -n flint-system get secret flint-gateway-token \
  -o jsonpath='{.data.token}' | base64 -d
```

That single value authenticates every REST call for every project. It is the **only** credential your harness needs — it never sees a per-share token and never needs bucket keys.

Rotating any of them

WHAT	HOW	RESTART?
Gateway token	edit the Secret; re-read every 10s	**no**
Per-share token	bump `chert.us/api-token-version`, rewrite that Secret	**no**
Bucket keys	rewrite `flint-s3`; label it `chert.us/credentials=true` so the operator notices at once	**no**
Gateway root key	rewrite, then re-derive **every** share's token	**gateway rollout**

The root key is the only fleet-wide rotation — every per-share token is a function of it, so change it least often.

2 Create a workspace

You write this resource yourself. The gateway can read and wake shares but is denied `create` and `delete` on purpose, so whatever owns projects decides a workspace exists.

```
kubectl -n workspaces create secret generic flint-s3 \
  --from-literal=AWS_ACCESS_KEY_ID=... --from-literal=AWS_SECRET_ACCESS_KEY=...
```

KEY NAMES ARE LOAD-BEARING

The secret is loaded with `envFrom`, so its **keys become environment variables verbatim**. Name one `accessKeyId` and the SDK has no credentials at all; the hub crash-loops with `tier: bucket posture refused: bucket <name> unreachable` — which names **the bucket, not the credentials**.

```
apiVersion: chert.us/v1alpha1
kind: FlintShare
metadata:
  name: fs-proj-a
  namespace: workspaces
  labels: { chert.us/project-id: proj-a }
spec:
  bucket: my-team-flint          # must exist, versioning ON
  keyPrefix: proj-a/             # immutable, must end in "/"
  credentialsSecretRef: flint-s3 # absent = IRSA / instance role
  persistence: { size: 50Gi }    # the CACHE, not the corpus
  monitoring:
    enabled: true
    fileApi: { enabled: true, tokenSecretRef: flint-api-token }
  settings: { hydrateFetchParallel: 6 }
```

One prefix, one volume, one hub. The operator refuses a second share on the same bucket subtree across every namespace — two live hubs on one prefix is a cross-tenant data leak the moment one is judged dead and the other takes over.

What the credentials must be allowed to do

A sensible-looking least-privilege policy is **not** enough. The hub refuses to start rather than run half-configured, and names the missing action:

```
✗ tier: bucket posture refused: s3:ListBucketMultipartUploads denied
  — the A9 startup sweep cannot run: 403 AccessDenied
```

SCOPE	ACTIONS
bucket	ListBucket · GetBucketLocation · GetBucketVersioning · ListBucketVersions · **ListBucketMultipartUploads** · GetLifecycleConfiguration
bucket/*	GetObject · PutObject · DeleteObject · GetObjectVersion · DeleteObjectVersion · AbortMultipartUpload · ListMultipartUploadParts

`ListBucketMultipartUploads` is **bucket-level**, which is what makes it easy to miss — every other multipart permission is object-level. A `cannot read lifecycle configuration` line is a *warning*, not an error: the hub starts, but cannot verify an MPU-abort rule exists.

3 Give the share its API token

The gateway holds no per-hub secrets — each hub's credential is derived from that share's identity, so one root key produces every credential and the gateway needs no access to Secrets in your workspace namespaces.

```
kubectl -n flint-system exec deploy/flint-lite-operator-gateway -- \
  flint-hub-gateway --root-key-file=/etc/flint/gateway-root/key \
  --derive-for workspaces/fs-proj-a

kubectl -n workspaces create secret generic flint-api-token --from-literal=token="<value>"
```

Prefer `--derive-for` to typing the binding. It reads the resource, so it cannot disagree with what the serving gateway computes. The manual form is the one step with no feedback: an omitted endpoint, a missing trailing slash, or a wrong version each yields a token that is perfectly valid, perfectly wrong, and rejected by every request.

Order does not matter — the token Secret is a projected volume, so a share created first waits in `ContainerCreating` and starts when the Secret lands. **A share does not reach `Ready` until its token exists**: if one is stuck, check the Secret before anything else.

4 Write through the REST API

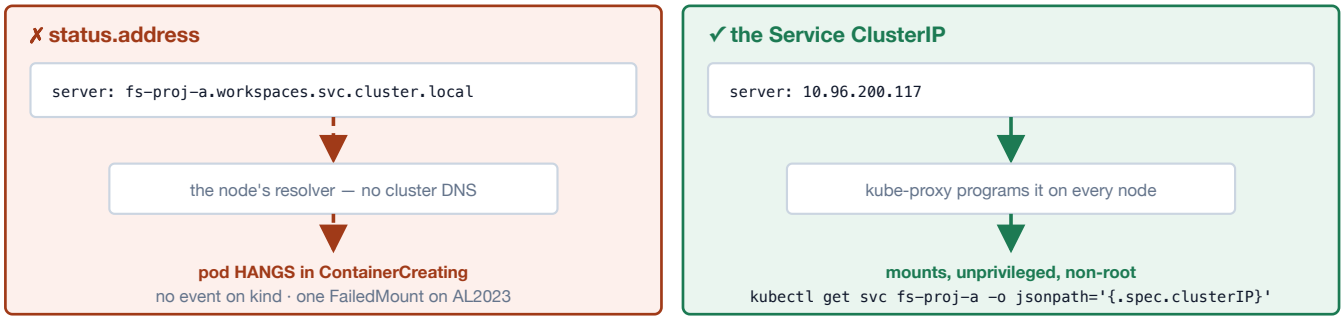
```
GET    /v1/projects/{id}/volumes
POST   /v1/projects/{id}/wake                → address, keepaliveSecs, mountWarning
GET    /v1/projects/{id}/files?path=&recursive=&cursor=
GET    /v1/projects/{id}/files/content?path=  [Range, If-None-Match]
PUT    /v1/projects/{id}/files/content?path=  [If-Match REQUIRED to replace]
DELETE /v1/projects/{id}/files/content?path=  [If-Match REQUIRED]
POST   /v1/projects/{id}/files/folder | /files/move
```

One `Authorization: Bearer` header. Every object carries an `ETag`: read it, edit, write back under `If-Match`, and a competing write answers `412` instead of silently losing your edit. Bodies **stream** in both directions — memory does not scale with file size. There is **no route to `/status`**, of any shape.

Creating needs no precondition; replacing or deleting does. An unconditioned `PUT` over a path that already exists, or an unconditioned `DELETE` of a file, is refused with **`428 Precondition Required`** — the server cannot tell an intentional overwrite from a caller who never saw the current version, so it refuses to guess. An absent path on `DELETE` stays a plain `404`. `If-Match: *` means "whatever is there now, but it must exist": right for a single-writer deploy replacing its own artefact, wrong on a user's behalf, because it re-admits the lost update. "Create or replace, either way" is two requests — `PUT` unconditioned, and on `428` re-issue under `If-Match: *`. **Which release:** the mandate is newer than every published image at the time of writing — `flint-pnfs:1.49.0` and earlier accept the unconditioned overwrite silently, and lose the update. Code written for the mandate is correct on both.

5 Read through a mounted PVC

An in-tree `nfs:` volume is resolved by KUBELET, ON THE NODE — not by the pod, and not by cluster DNS.



The address is the whole game. The ClusterIP is stable for the life of the Service — suspend and hibernate both keep it, only deleting the share releases it. For agents in **another cluster or on-premises**, publish a LoadBalancer or NodePort address with `spec.service.advertiseAddress` instead; a bare host is refused at admission, because an NFS client handed one silently uses 2049.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata: { name: proj-a }
spec:
  capacity: { storage: 100Gi }        # informational for NFS
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  mountOptions: [nfsvers=4.1, proto=tcp, hard, nconnect=4, noatime, sec=sys]
  nfs:
    server: 10.96.200.117             # the ClusterIP, not the DNS name
    path: /                           # the SERVER ROOT
```

Agent pods then mount the claim like any other volume — **no privilege, no root, no extra images**, with `runAsUser` / `runAsGroup` / `fsGroup` set. Kubelet mounts once per node, so pods on one node share a kernel client and page cache.

TWO MOUNT OPTIONS THAT FAIL SILENTLY

`nconnect>=2` — without it the kernel opens exactly one connection and silently refuses every additional trunk. No error, on either side, ever.

`sec=sys` — without it the kernel negotiates `sec=null` and sends **no credential at all**. Measured on the same server, one option apart:

	SEC=NULL (the default you get)	SEC=SYS
File created by a uid-1000 pod	`owned 0:0`	`owned 1000:1000`
Non-root pod writing into root-owned 0755	**succeeds**	**obeys the mode**
Effective identity of every client	**root**	**the pod's own uid**

The mount carries **no credentials** and there is no root-squash: identity is whatever the client asserts. **Reachability is therefore the access control** — anything that can route to port 2049 can claim any uid, so keep hubs on the cluster network and use `networkPolicy` to name your agent pods.

NFSCLIENTCIDRS CANNOT REACH ACROSS CLUSTERS

Nothing sets `externalTrafficPolicy`, so kube-proxy SNATs a NodePort or LoadBalancer client to an address in the **hub's own** cluster before the packet reaches the pod. Captured on a three-cluster rig: **1486 of 1486** connections from two remote clusters arrived as the hub cluster's gateway, none as either remote node. Listing the remote CIDRs denies the address the traffic actually comes from — an outage; listing the local one admits everyone who can reach the NodePort. **Bound cross-cluster reach at the network layer** — peering, a security group, or a gateway. The selectors work as written for clients inside the hub's cluster, which arrive as themselves.

Cold reads: what an agent experiences

At the disk watermark (default 85%) cold files truncate to stubs. Metadata stays truthful — `ls -l` and `df` show logical sizes — so a tree looks complete whether or not it is local. The first read parks the client until the **whole file** restores; kernel clients retry silently and the application just sees a slow open. Restores issue up to `hydrateFetchParallel` (default 6) ranged GETs concurrently.

VERIFIED END TO END

With the file confirmed gone from local disk — the hub's own copy at **0 bytes, 0 blocks**, the data only in the bucket — an agent read it back through the mount **byte-identical**, with no error and nothing in its way.

If your agents sweep the tree (`grep -r`, a build, a full index), set `hydrateWarmAfterImport: true` and the hub bulk-restores every stub after an import, smallest first. Off by default, because a fill re-downloads every byte.

One hub, many clusters: give every client a unique name

The common shape for an agent fleet is several clusters mounting **one** hub. Nothing about that is unusual, and nothing about it is safe by default — because on NFSv4.1 the client identifies itself as `Linux NFSv4.<minor> <nodename>` and **nothing else**. No address, no cluster, no uniquifier. Two clusters built from one manifest hand their nodes the same names, and two clusters on the same VPC CIDR derive the same hostnames from the private IP.

CAPTURED ON THE WIRE FROM TWO CLUSTERS MOUNTING ONE HUB

```
cluster B → Linux NFSv4.2 agent
cluster C → Linux NFSv4.2 agent
```

Byte-identical. The server is **required** to read the second one as the first one rebooting, so it discards the first client's session and open state. It says so, but at `info`, and it reads like housekeeping: `EXCHANGE_ID: case 5 (client reboot detected)`.

Which name matters depends on who mounts. A `PersistentVolume` is mounted by kubelet in the node's namespace, so the identity is the **node's** hostname; a pod that runs `mount` itself uses the **pod's**. Whichever applies has to be unique across every cluster that mounts this hub — not merely within one.

Two ways there. **Name the nodes uniquely**, including the cluster name in the hostname. Or **set the uniquifier**: `nfs.nfs4_unique_id=<something-unique>` as a kernel module parameter on each node, a drop-in under `/etc/modprobe.d/` applied at node bootstrap. Read it back at `/sys/fs/nfs/net/nfs_client/identifier` — `(null)` means it is not set and the hostname is doing all the work. To audit a running fleet, count the distinct identities your clients actually send; fewer than the number of clients means a collision.

A SECOND REASON, WHICH ONLY SHOWS UP WHEN SOMETHING GOES WRONG

State loss is the obvious hazard. The subtler one is that a shared name breaks the server's ability to **tell you** about a loss. When a lease expires the server may release that client's locks — deliberate, and what lets a partitioned cluster's ranges become available again. It reports that as a status flag on the next `SEQUENCE`, and that flag is addressed to a **client id**, not to a cluster. Two clusters sharing one identity share one client id, so whichever cluster's traffic arrives first consumes the notification, and the cluster that actually lost the range never hears about it — carrying on believing it holds a lock the hub has already handed to someone else.

If you enable idle-suspend, a remote mount needs a keepalive

The idle ladder scales a quiet hub to zero so it stops costing anything. That is safe when the consumer is in the same cluster and unsafe when it is not, and the reason is worth understanding rather than working around.

`suspendWithSessions: false` looks like the answer — it refuses to suspend while any client holds a lease. It narrows the window; it does not close it. **A lease expires.** A cluster that is partitioned rather than gone stops renewing, the count reaches zero on its own, and the guard stops guarding.

MEASURED ACROSS A ONE-WAY PACKET CUT

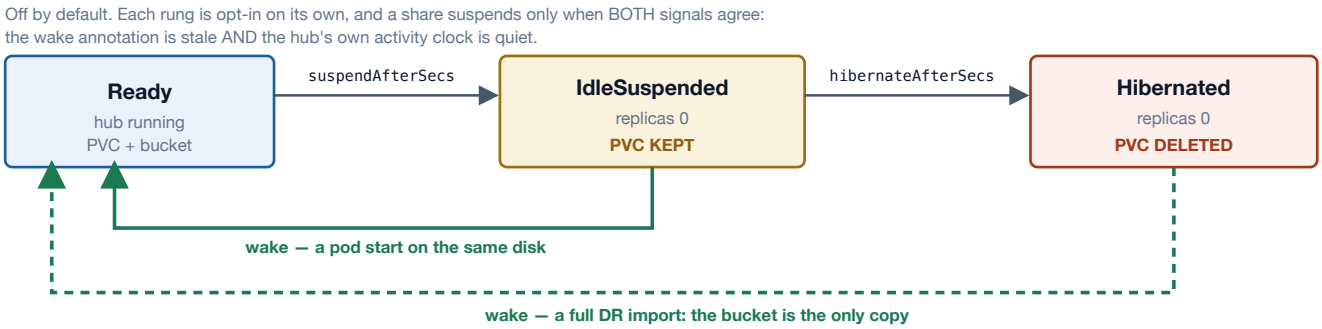
The guard held from **t=49 to t=99** reporting "a client still holds a lease"; the lease lapsed **1 → 0 at t=99**; the share **suspended at t=111** with the mount still held. A control share — connected, quiet, at 3.4x the idle threshold — never suspended, which is what makes that attributable to the partition and not to the clock.

Healing the partition does not recover it. The mount stays hung, and the wake is an annotation on a custom resource in the **hub's** API server — which a workload cluster has no credential for, and no copy of the CRD.

The remedy is a keepalive on a different path from the mount. Deploy `flint-hub-gateway` and have your harness call `POST /v1/projects/{id}/volumes/{v}/wake` every `keepaliveSecs` while a mount is held. It needs one bearer token and no Kubernetes credential, it stamps the request on every call so it wakes *and* keeps alive, and because it crosses a different network path from the mount it still arrives when the mount's path is cut. The gateway is off by default, so this is something you turn on deliberately.

If you cannot run that keepalive, leave the ladder off for shares mounted from another cluster. A hub that stays up costs money; a hub that suspends under a remote mount costs an outage nothing in that cluster can clear.

6 The workspace lifecycle



Suspend and hibernate are not one setting with two numbers. Suspend keeps the PVC, so waking is seconds. Hibernate deletes it, so the bucket becomes the only copy and waking is a full restore — which is why it requires `spec.bucket` and is verified before the disk is destroyed. **The CR is never deleted by either.**

```yaml
spec:
  idle:
    suspendAfterSecs: 900            # 15m quiet → scale to zero, keep the PVC
    suspendWithSessions: false       # the protective value is OPT-IN
```

AN AGENT THAT THINKS FOR TWENTY MINUTES TRIPS BOTH SIGNALS

It touches nothing on the filesystem, so the hub sees an idle share; nothing stamps the wake annotation, so the front door looks absent. The hub scales to zero underneath a live `hard` mount — and a `hard` mount whose server is gone blocks in uninterruptible sleep. **An NFS client cannot write a Kubernetes annotation, so nothing wakes it.** Not data loss; an indefinite hang.

Set `suspendWithSessions: false` *and* heartbeat with `POST /wake`, which re-stamps on every call even when the share is already `Ready` and tells you how often to call it:

FIELD IN THE WAKE REPLY	WHAT TO DO WITH IT
`keepaliveSecs`	the interval to call `/wake` at — half the suspend budget, so one missed beat survives
`mountWarning`	present exactly when this share can be scaled to zero under a live mount. A provisioning error, not advice
`serverId`	changes after a hibernate ⇒ remount, not resume. Absent means *not observed*, never *changed*

Do **not** heartbeat by polling files: `/status` is deliberately not activity and the file API deliberately is, so a UI refreshing a listing on a timer pins that project awake forever. For a fleet crawl that must not start thousands of hubs, add `?wake=false` — it refuses with `503 Parked` instead of waking anything.

Tearing a workspace down

Unmount before you remove the server. A `hard` mount whose server disappears puts clients into uninterruptible sleep; those processes cannot be killed, and on a node this can wedge the container runtime itself. Scale consumers to zero, confirm nothing holds the claim, *then* delete the `FlintShare`. If you are already wedged, bring the **server** back — the blocked I/O completes and the processes exit normally.

7 Cloud and on-premises

Everything above is identical everywhere. Four things differ by platform, and all four live in the `FlintShare`.

WHAT	SET IT TO
Hub cache disk	`persistence.storageClassName: gp3` / *`csi-cinder-high-speed` / `rook-ceph-block`*
AWS with IRSA	omit `credentialsSecretRef` entirely — absent means ambient (env / IRSA / IMDS)
Ceph RGW, MinIO	`spec.endpoint: https://rgw.corp.internal:8443` *`# forces path-style`*
Reaching the hub	same cluster → Service ClusterIP; elsewhere → LoadBalancer / NodePort via `spec.service.advertiseAddress`

`spec.endpoint` is what makes a non-AWS store work, and **it participates in the share's identity** — it is part of the derived API token, so changing it invalidates every token for that share. Prefer a flat or peered network to a load balancer: NFS is one long-lived TCP flow and quiet

periods are normal, so an LB that reaps idle connections breaks mounts that were working. Keep hub and agents in one availability zone where you can; inter-AZ bytes are billed in both directions.

What the two doors guarantee about each other

They are one filesystem — the REST API dispatches in-process through the same server the mount talks to. The timing is NFS timing:

- **A REST write shows up on an established mount immediately**, as long as the agent opens the file to look. Measured under a second for both a new file and an overwrite: every `open()` revalidates with the server, so the attribute cache does not sit between your harness and your agents.
- Staleness is real in two cases: **a file the agent holds open** across the write, and **a cached directory listing** (up to `acdirmax`, 60s).
- **`If-Match` is detection between API callers, not exclusion against the mount.** Two REST writers get `412`; a REST writer racing a mounted writer is ordered by the server but the ETag will not save you.
- An **ETag changes when a file is evicted or hydrated**, because both rewrite the local inode. Treat a lone `412` as "re-read before you write".

What this is not

Not a parallel filesystem	One hub is one pod and one NIC. If a single pod's bandwidth is your ceiling, the full pNFS profile speaks the same protocol and stores the same volume format — graduating is adding data servers, not migrating.
Not multi-writer across regions	Keep hub and agents in one AZ where you can.
Not a bucket manager	Create the bucket, turn versioning on, hold the lifecycle policy yourself.
Not a place for secrets	Anything that can reach 2049 can read the tree as any uid.

How this guide was verified

Every instruction here was executed against a real cluster by `tests/regression/agent-fleet-doc-drill.sh`, which runs the guide's own commands rather than a paraphrase. Each claim is paired with a control that would fail if the product were broken — a leg that only shows the good case proves nothing.

CLAIM	ITS CONTROL
Wrong `AWS_*` key names never reach Ready	a share built on `accessKeyId` crash-loops
Nothing in the YAML is silently pruned	every field read back after admission
A `.svc.cluster.local` server does not mount	pod stuck Pending; the ClusterIP mounts in the same leg
`sec=sys` is load-bearing	same pod, one option removed ⇒ 0:0 instead of 1000:1000
REST writes reach a live mount	new file and overwrite, both under a second
Eviction really hydrates from S3	the hub's local copy at 0 bytes / 0 blocks before the read
`suspendWithSessions: false` holds a share up	an unprotected control share **did** suspend under a live mount